Task: Mobile VAPT Report
Name: Kavita Prajapati

Executive Summary

Mobile Application Vulnerability Assessment and Penetration Testing (VAPT) is the process of identifying, analyzing, and mitigating security weaknesses in mobile applications across Android and iOS platforms. It involves a combination of static and dynamic analysis to evaluate how an application handles sensitive data, authentication, session management, and communication with backend services.

The assessment focuses on identifying vulnerabilities such as insecure data storage, improper authentication and authorization, insecure network communication, reverse engineering risks, and exposure of sensitive information. Testing is performed using industry-standard methodologies like the **OWASP Mobile Top 10**, along with tools such as **Burp Suite**, **MobSF**, and **Frida**.

CVSS Scoring System:

The Common Vulnerability Scoring System (CVSS) provides a standardized method for assessing vulnerability severity. Security analysts often use the CVSS calculator provided by the National Vulnerability Database to determine vulnerability scores.

Score Range	Risk Level
0 – 3.9	**Low**
4 – 6.9	**Medium**
7 – 8.9	**High**
9 – 10	**Critical**

Risk Matrix:

A risk matrix is another method used to prioritize vulnerabilities based on their likelihood and impact. EX:,

Vulnerability	Likelihood	Impact	Risk Level
SQL Injection	High	High	Critical
XSS	Medium	Medium	Medium
Open Ports	Low	Medium	Low

Information:

Item	Details
Project Title	Mobile Vulnerability Assessment of Kali Linux Environment
Assessment Type	Vulnerability Assessment Lab
Tools Used	Kali Linux, Mobsf
Testing Environment	VirtualBox Lab
Date	03/04/2026

Executive summary:

The objective of this assessment was to identify security vulnerabilities in a controlled testing environment using open-source security tools. The assessment was performed on a vulnerable virtual machine to simulate a real-world security evaluation scenario.

The scanning process identified several vulnerabilities including outdated software versions, exposed services, and configuration weaknesses. These issues could potentially allow attackers to gain unauthorized access to the system if not properly mitigated. Risk assessment was performed using CVSS scoring to prioritize remediation efforts.

Tools Used:

Tool	Purpose
Kali Linux	Security testing platform
MobSf	Vulnerability Assesment
Oracle VM VirtualBox	Virtual lab environment

The below table gives a brief description of each risk rating visualized in this report.

Impact	CVSS Score	Definition
Critical	9.0 – 10.0	Critical vulnerabilities are the highest level of security risks, demanding immediate attention and mitigation within an organization. They pose an imminent and severe threat, with the potential for unauthorized access, data breaches, or significant system compromise. Swift remediation is crucial to maintain robust security.
High	7.0 – 8.9	High severity vulnerabilities should be considered a top priority in terms of mitigation. These are the most severe issues and generally cause an immediate security concern to the enterprise
Medium	4.0 – 6.9	Medium severity vulnerabilities are a lower priority but should still be remediated in a timely manner. These are moderate exploits that have less of an impact on the environment.
Low	0.1 – 3.9	Low severity vulnerabilities are real but trivially impactful to the environment. These should only be remediated after the HIGH and MEDIUM

		vulnerabilities are resolved.
Informational	0.0	Informational vulnerabilities have no impact as such to the environment by themselves. However, they might provide an attacker with information to exploit other vulnerabilities.

The following control areas are the easiest to exploit or expose the environment to the highest level of risk:

Sr. No.	Vulnerability Name	Severity Level
1	Insecure Storage of Sensitive Data in Android AccountManager	**High**
2	Improper Authentication Token Validation in AccountAuthenticator	**Medium**

Vulnerabilities Details:

Sr.No. 1	**Name of Vulnerability:** Insecure Storage of Sensitive Data in Android AccountManager	**CWE Details:** CWE-922
High	**CVSS Score:** 7.8	
Vulnerable URL: Netmirror.apk		
Description: Insecure Storage of Sensitive Data in Android AccountManager		
Impact: The application is storing sensitive user information such as: • access_token • refresh_token • store password		

- user details (username, avatar, etc.)

PoC:

```
import android.accounts.AccountManager;
import cm.aptoide.accountmanager.Account;
import cm.aptoide.accountmanager.AccountFactory;
import cm.aptoide.accountmanager.AccountPersistence;
import cm.aptoide.accountmanager.Store;
import cm.aptoide.pt.crashreports.CrashReport;
```

```
this.androidAccountManager.setUserData(account2, ACCOUNT_ID, account.getId());
this.androidAccountManager.setUserData(account2, "username", account.getNickname());
this.androidAccountManager.setUserData(account2, ACCOUNT_AVATAR_URL, account.getAvatar());
this.androidAccountManager.setUserData(account2, ACCOUNT_ADULT_CONTENT_ENABLED, String.valueOf(account.isAdultContentEnabled()));
this.androidAccountManager.setUserData(account2, ACCOUNT_ACCESS_LEVEL, account.getAccess().name());
this.androidAccountManager.setUserData(account2, ACCOUNT_ACCESS_CONFIRMED, String.valueOf(account.isAccessConfirmed()));
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_NAME, account.getStore().getName());
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_AVATAR_URL, account.getStore().getAvatar());
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_DOWNLOAD_COUNT, String.valueOf(account.getStore().getDownloadCount()));
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_ID, String.valueOf(account.getStore().getId()));
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_THEME, account.getStore().getTheme());
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_USERNAME, account.getStore().getUsername());
this.androidAccountManager.setUserData(account2, ACCOUNT_STORE_PASSWORD, account.getStore().getPassword());
this.androidAccountManager.setUserData(account2, ACCOUNT_TERMS_AND_CONDITIONS, String.valueOf(account.acceptedTermsAndConditions()));
this.androidAccountManager.setUserData(account2, ACCOUNT_PRIVACY_POLICY, String.valueOf(account.acceptedPrivacyPolicy()));
```

```
import android.accounts.Account;
import android.accounts.AccountManager;
import cm.aptoide.pt.account.AndroidAccountProvider;
import okhttp3.HttpUrl;
import rx.Single;

/* loaded from: classes.dex */
public class AuthenticationPersistence {
    public static final String ACCOUNT_ACCESS_TOKEN = "access_token";
    public static final String ACCOUNT_REFRESH_TOKEN = "refresh_token";
    public static final String ACCOUNT_TYPE = "aptoide_account_manager_login_mode";
    private final AccountManager androidAccountManager;
    private final AndroidAccountProvider androidAccountProvider;

    public AuthenticationPersistence(AndroidAccountProvider androidAccountProvider, AccountManager accountManager) {
        this.androidAccountProvider = androidAccountProvider;
        this.androidAccountManager = accountManager;
    }

    /* JADX INFO: Access modifiers changed from: private */
    /* renamed from: lambda$createAuthentication$3, reason: merged with bridge method [inline-methods] */
    public /* synthetic */ void a(String str, String str2, String str3, String str4, Account account) {
        this.androidAccountManager.setUserData(account, ACCOUNT_REFRESH_TOKEN, str);
        this.androidAccountManager.setUserData(account, "access_token", str2);
        this.androidAccountManager.setPassword(account, str3);
        this.androidAccountManager.setUserData(account, ACCOUNT_TYPE, str4);
```

Recommendation:

- Do NOT store sensitive data in AccountManager
- Use Secure Storage
- Encrypt Sensitive Data
- Root Detection

Reference: https://radar.offseq.com/threat/cve-2025-21041-cwe-922-insecure-storage-of-sensiti-4236ea8a?utm_source

Sr.No. 2	**Name of Vulnerability:** Improper Authentication Token Validation in AccountAuthenticator	**CWE Details:** CVE-2025-0086
Medium	**CVSS Score:** 5.3	

Vulnerable URL: Netmirror.apk

Description:

- Unauthorized token requests
- Improper authentication flow handling
- Potential abuse of account authentication mechanism

Impact:

- Authentication bypass (in weak implementations)
- Unauthorized token generation or misuse
- Abuse of AccountManager authentication flow
- Exposure of protected resources

PoC:

```
import android.accounts.AccountAuthenticatorResponse;
import android.accounts.AccountManager;
import android.accounts.NetworkErrorException;
```

```
@Override // android.accounts.AbstractAccountAuthenticator
public Bundle getAuthToken(AccountAuthenticatorResponse accountAuthenticatorResponse, Account account, String str, Bundle bundle) throws NetworkErrorException {
    if (!str.equals(AUTHTOKEN_TYPE_FULL_ACCESS)) {
        Bundle bundle2 = new Bundle();
```

Recommendation:

- Strict Token Type Validation
- Return Proper Error Response
- Follow Secure Auth Design
- Logging & Monitoring

Reference: https://nvd.nist.gov/vuln/detail/CVE-2025-0086?utm_source=feedly